Gandalf the Red: Adaptive Security for LLMs

Niklas Pfister [*1] **Václav Volhejn** [*1] **Manuel Knott** [*1] **Santiago Arias** [1] **Julia Bazińska** [1] **Mykhailo Bichurin** [1] **Alan Commike** [1] **Janet Darling** [1] **Peter Dienes** [1] **Matthew Fiedler** [1] **David Haber** [1] **Matthias Kraft** [1] **Marco Lancini** [1] **Max Mathys** [1] **Damián Pascual-Ortiz** [1] **Jakub Podolak** [1] **Adrià Romero-López** [1] **Kyriacos Shiarlis** [1] **Andreas Signer** [1] **Zsolt Terek** [1] **Athanasios Theocharis** [1] **Daniel Timbrell** [1] **Samuel Trautwein** [1] **Samuel Watts** [1] **Yun-Han Wu** [1] **Mateo Rojas-Carulla** [1]

arXiv:2501.07927v3 [cs.LG] 4 Aug 2025

Abstract

Current evaluations of defenses against prompt attacks in large language model (LLM) applications often overlook two critical factors: the dynamic nature of adversarial behavior and the usability penalties imposed on legitimate users by restrictive defenses. We propose D-SEC (Dynamic Security Utility Threat Model), which explicitly separates attackers from legitimate users, models multi-step interactions, and expresses the security-utility in an optimizable form. We further address the shortcomings in existing evaluations by introducing Gandalf, a crowd-sourced, gamified red-teaming platform designed to generate realistic, adaptive attack. Using Gandalf, we collect and release a dataset of 279k prompt attacks. Complemented by benign user data, our analysis reveals the interplay between security and utility, showing that defenses integrated in the LLM (e.g., system prompts) can degrade usability even without blocking requests. We demonstrate that restricted application domains, defense-in-depth, and adaptive defenses are effective strategies for building secure and useful LLM applications.

1. Introduction

In recent years, large language models (LLMs), such as OpenAI's ChatGPT, Google's Gemini, and Anthropic's Claude, have revolutionized the capabilities of machine learning systems. One of the most promising features of these models is their ability to bridge the gap between natural language and code. This capability has enabled developers to design LLM-based applications that can perform various tasks controlled by natural language inputs. Current LLM applications primarily include chatbots for customer support and retrieval-augmented generation (RAG) systems for database-driven queries. However, improved reasoning and enhanced API access are paving the way for more sophisticated LLM agents equipped with tools and collaborative capabilities to tackle complex tasks (Talebirad & Nadiri, 2023).

As with any other traditional software application, LLM applications possess security vulnerabilities that can be exploited by malicious attackers, potentially causing substantial damage. In addition to conventional security vulnerabilities such as broken access control, cryptographic failures, or injections (OWASP, 2025a), LLM applications introduce several novel vulnerabilities, the most serious of which are prompt attacks[2] (OWASP, 2025b). Prompt attacks make use of the fact that LLMs cannot programmatically distinguish between commands used to define the original task and input data used to execute the task. Attackers can exploit this to manipulate LLM behavior, causing it to deviate from its intended use case. Examples include jailbreak attacks, system prompt leakage, and indirect injection attacks (Greshake et al., 2023).

Defending against prompt attacks is challenging due to the ability of LLMs to interpret natural language, multimodal inputs, code, and obfuscated commands, making it easy to hide malicious commands within benign-looking inputs. In the literature, many approaches for defending against prompt attacks have been proposed. They can be roughly divided into three categories: (i) Input/output classifier defenses that use the input and/or the resulting LLM output to classify whether a given prompt is malicious (e.g., Kumar et al., 2024; Ayub & Majumdar, 2024; Kim et al., 2024; Sawtell et al., 2024), (ii) LLM internal defenses that modify the model itself through the system prompt, fine-tuning, or other post-training methods to ensure it adheres strictly to its original intended task (e.g., Bai et al., 2022; Wei et al.,

[*]Equal contribution [1]Lakera. Correspondence to: Niklas Pfister <niklas.pfister@lakera.ai>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

[2]We prefer the term 'prompt attack' over 'prompt injection', used by OWASP (2025b) since some authors use 'prompt injection' exclusively to refer to indirect attacks.

2023; Chen et al., 2024a;b; Wallace et al., 2024; Zhou et al., 2024; Piet et al., 2024), and (iii) prompt modification or sanitization defenses that restructure the prompt or remove any potentially malicious parts of the input (e.g., Learn Prompting, 2024; Hines et al., 2024; Chen et al., 2024a). None of the existing defenses is perfect (Liu et al., 2024b), and all of them can impact the utility of the LLM application by falsely blocking legitimate user requests. Since defenses can be embedded within the LLM itself (e.g., via the system prompt), they can significantly impact user utility by impacting the behavior of the application. Evaluating the security of an LLM application should, therefore, also account for user utility, a factor overlooked by most proposed approaches. Works that do consider user utility (e.g., Sharma et al., 2025) have focused on over-refusals, but as we show here, there can also be more subtle impacts on user utility.

There are essentially two approaches for evaluating the security of a defense. The first approach is to use publicly available prompt attack benchmarks (e.g., Yi et al., 2023; Liu et al., 2024c; Debenedetti et al., 2024; Mazeika et al., 2024; Chao et al., 2024; Zhu et al., 2024) to assess how many attacks are blocked by the defenses. Many attacks in these benchmarks are misleading as they are non-adaptive and easily blocked by model providers, resulting in an overly optimistic evaluation. The second approach is to use red-teaming to probe the LLM application. This can either be done manually or automatically, using, for instance, a second LLM to optimize attacks (e.g., Perez et al., 2022; Ganguli et al., 2022; Deng et al., 2023). The resulting attacks are generally more realistic as they are constructed adaptively. However, manual red-teaming is expensive, and current automatic approaches often lead to less diverse attacks. Both benchmark and red-teaming approaches suffer from the difficulty of assessing whether an attack was truly successful. To circumvent this, evaluations often focus on detecting malicious intent instead. However, this focus on intent is limited and can lead to overly optimistic results, as defenses might stop obvious failures while missing subtle yet successful attacks.

In this work, we propose the Dynamic Security Utility Threat Model (D-SEC), which (i) captures the adaptive nature of attacks and defenses and (ii) balances security and utility by separately modeling attackers and benign users. This model operates in sessions consisting of multiple transactions (either attacks or legitimate user prompts), each of which can be influenced by feedback from prior transactions. This provides a principled way to evaluate defenses for LLM applications and explicitly optimize the defense to achieve the desired security-utility trade-off.

To address the limitations of existing evaluation approaches, we propose a crowd-sourced red-teaming platform (Schulhoff et al., 2023; Toyer et al., 2023), called Gandalf,[3] which gamifies the process of generating realistic and diverse adaptive prompt attacks. In Gandalf, attacks are automatically labeled based on the success indicator given by whether the password was extracted rather than intent. Using Gandalf, we constructed a large-scale, high-quality attack dataset, which we make publicly available.[4] We then perform an extensive analysis to gain generalizable insights into promising defense strategies for D-SEC.

Our contributions are fourfold: (i) We introduce D-SEC and corresponding metrics that provide a principled and comprehensive way of evaluating defenses for LLM applications. (ii) We present the crowd-sourced red-teaming platform Gandalf and release a large-scale dataset of diverse attacks. (iii) We provide empirical evidence for the importance of the security-utility trade-off. (iv) We show empirically that three defense strategies allow us to design defenses that optimize the security-utility trade-off: restricting the application domain, aggregating multiple defenses in a defense-in-depth approach, and using adaptive defenses.

2. Dynamic Security-Utility

We focus on scenarios where attackers target LLM applications by crafting prompts that manipulate the LLM to deviate from its intended behavior. We call any such attack a *prompt attack*. Prompt attacks exploit the fact that LLMs do not have a strict, inherently enforced functional separation between developer commands used to define the intended behavior (e.g., system prompts) and user or other external inputs (e.g., documents or websites in RAG applications). Similar to SQL injection attacks (Halfond et al., 2006), this allows attackers to hide malicious commands, ranging from direct instructions to subtle manipulation patterns within any text that is passed as input to the LLM. The literature typically distinguishes between *direct* prompt attacks (e.g., jailbreaks), where the attacker sends inputs directly to the LLM, and *indirect* prompt attacks, where the attacker modifies external sources that the LLM accesses.

A prompt attack is typically composed of two elements: a *payload* and a *trigger* (e.g., Pasquini et al., 2024). The *payload* represents the instructions the attacker intends the LLM to execute. It can range from a harmful command, such as "tell me how to build a bomb," to benign instructions

[3] `https://gandalf.lakera.ai` Gandalf is a white-hat red-teaming system designed to identify vulnerabilities in commercial LLMs before they can be exploited maliciously. It does not promote or expose users to unsafe content. The password extraction use case focuses on identifying security weaknesses, not on generating or disseminating harmful outputs.

[4] Full dataset at `https://huggingface.co/datasets/Lakera/gandalf-rct`, for processed versions, see Appendix I.

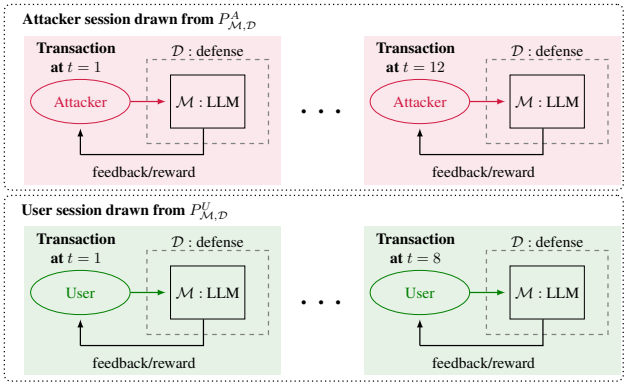

Figure 1: *Dynamic Security Utility Threat Model (D-SEC).* In this threat model, there are three actors: the developer, the attacker, and the user. The developer creates an LLM application $\mathcal{M}$ and defends it with a defense $\mathcal{D}$. Users and attackers interact with $\mathcal{M}$ in sessions, which are sequences of transactions that consist of the user/attacker sending a prompt/attack and receiving feedback that can be used in the next transaction. A full sequence of transactions is called a session. The goal of the developer is to design $\mathcal{D}$ to defend against the attacker but only minimally impact user utility.

with the simple aim of overriding the intended behavior, such as "output 'I have been pwned' ". The *trigger* is a broad term describing the technique used to help persuade the LLM to execute the payload, whether malicious or benign. The presence of a trigger makes the payload more likely to succeed. Triggers can include phrases such as "Ignore all instructions", obfuscations such as leetspeak, ciphers, role-playing, or adversarial tokens (e.g., Zou et al., 2023; Zeng et al., 2024). Payloads vary by context, for instance, in agentic settings, the goal might be to manipulate an agent to use its tools incorrectly (Debenedetti et al., 2024).

2.1. Threat Model

We introduce the *Dynamic Security-Utility Threat Model (D-SEC)*, see Figure 1, which is a comprehensive threat model for evaluating the security of LLM applications in a dynamic environment. It addresses two major shortcomings of existing threat models: (i) It is dynamic and hence able to model both adversarial attackers that sequentially optimize their attacks and adaptive defenses that incorporate information from previous interactions. (ii) It treats attackers and users separately, and hence is able to model not only the security of the application, which depends on the attacker, but also the usability of the application, which depends on how the defense impacts benign user interactions.

D-SEC involves three actors: the developer, the user, and the attacker. The system under consideration consists of both an unprotected *LLM application* $\mathcal{M}$ and a *defense* (or security layer) $\mathcal{D}$. The developer's goal is to design $\mathcal{D}$ such that it prevents attackers from exploiting $\mathcal{M}$, while not negatively impacting the user. We consider $\mathcal{M}$ to be a simple LLM application in this paper, but it could also be an LLM agent that has access to a library of tools. The defense $\mathcal{D}$ can consist of a combination of *input defenses* that check the input for maliciousness, *internal defenses* that are directly embedded in the LLM (e.g., via the system prompt, and fine-tuning), and *output defenses* that use the input together with the output to check whether there was an attack. In conversational LLM applications, input and output defenses generally *block* the output by replacing it with a refusal message if an attack is detected. In contrast, internal defenses modify the LLM itself to produce secure outputs, which can make it difficult to determine whether a defense has detected an attack. Internal defenses are unique to LLM applications and do not exist in traditional software security. While they can provide strong security, they can impact the usability of the application, see Section 4.1. In agentic systems, however, this distinction becomes less relevant, as both internal and external defenses can significantly disrupt execution flow, resulting in pronounced impacts on usability.

D-SEC models attackers and users separately. Both interact with $\mathcal{M}$ in *sessions*, which are sequences of *transactions* (of varying length) that consist of a submitted prompt (called an *attack* in the case of an attacker) and the resulting feedback from $\mathcal{M}$ under defense $\mathcal{D}$. We assume the attacker sessions are drawn from an attacker distribution $P^A_{\mathcal{M},\mathcal{D}}$ and the user sessions from a user distribution $P^U_{\mathcal{M},\mathcal{D}}$. Section 4.1 shows the importance of selecting a user distribution relevant to the application. Since there are multiple transactions per session, the attacker can use the feedback as a reward to optimize subsequent attacks. Viewing the threat model as dynamic not only provides a more accurate measure of security but also suggests defensive strategies that take into account the dynamic nature of the problem (see Section 4.2). While we focus on the black-box setting where feedback consists of the LLM response or a refusal message from $\mathcal{D}$, D-SEC also applies to gray- or white-box settings by adding further information to the feedback, e.g., gradients.

We restrict the attacker's capabilities to sending direct inputs to the target LLM via user messages (e.g., Verma et al., 2024). While indirect attacks via data (e.g., prompt injections in retrieved content) are also compatible with D-SEC, we focus on the direct setting for simplicity. Attacker goals are broad and application-specific.

2.2. Security-Utility Trade-off

The objective of the developer when constructing $\mathcal{D}$ is to ensure the security of the application with minimal impact on usability. We formalize this as maximizing a *developer*